University of Pisa

Master's degree in Artificial Intelligence and Data Engineering

Project for

ICT Risk Assessment

A Rule-Based Phishing Email Risk Analyzer

Header, Link and Language Signals for Explainable Risk Scoring

`https://github.com/Ahmed-Zarrad/ICTRA-Project`

Professor:
FABRIZIO ENRICO
ERMINIO BAIARDI
LUCA DERI

Student:
Ahmed Zarrad
ID: 722204

Academic Year 2025/2026

Contents

Contents **1**

1 Abstract **2**

2 Introduction **2**

3 Threat Model and Risk Indicators **3**

4 Datasets **3**

5 Overall Architecture **4**
5.1 Shared data model 4

6 Module 1: Email Parsing **5**

7 Module 2: Header Analysis **5**
7.1 The fairness rule 5
7.2 Sender-identity checks 6
7.3 List/bulk awareness 6

8 Module 3: Link Analysis **6**
8.1 Look-alike (typosquatted) domains 6
8.2 Other link signals 7

9 Module 4: Language Analysis **7**

10 Module 5: Risk Scoring **8**

11 Module 6: Reporting **8**

12 Experimental Evaluation **9**
12.1 Interpreting the confusion matrix and the accuracy figure 10
12.2 Per-module contribution 12
12.3 Choosing the operating point 12

13 Discussion **12**
13.1 Fairness constraints are load-bearing 12
13.2 Limitations 13

14 Conclusions **13**

1 Abstract

This project implements a **rule-based** (non-machine-learning) phishing email risk analyzer written in pure Python 3 with **no third-party dependencies**. Given an email file, the tool assesses its phishing risk from three complementary angles (header authentication, link destinations, and social-engineering language) and combines them into a single weighted risk score and a categorical band (*low*, *medium*, *high*, *critical*), accompanied by an explainable HTML report in which every point of the score is traceable to a concrete finding. The system is evaluated on two real corpora: the *Phishing Pot* dataset (8,608 phishing emails with full headers) and the SpamAssassin *ham* dataset (2,750 legitimate emails used as a control set). The central design priority, consistent with a risk-assessment perspective, is a **low false-positive rate**: on the full corpus the analyzer achieves a detection rate of **67.1%** at a false-positive rate of only **0.44%** (precision 99.8%, $F_1 = 80.2\%$), with no legitimate email ever rated *high* or *critical*.

2 Introduction

Phishing remains one of the most common and damaging initial-access techniques in modern cyber attacks: an adversary impersonates a trusted entity to lure a victim into revealing credentials, opening a malicious link, or authorising a fraudulent transaction. From a risk-assessment standpoint, the relevant question is not only *whether* an email is phishing, but *which observable indicators* contribute to that judgement and *how confident* the assessment is. An analyzer that produces an opaque verdict is of limited value to a security analyst, whereas one that explains its reasoning supports triage and prioritisation.

This work deliberately adopts a **rule-based** approach rather than a machine-learning classifier. The motivations are pedagogical and practical: rule-based scoring is fully transparent (each contribution to the risk score is a named, human-readable rule), it requires no training data or model artefacts, and it makes the underlying risk indicators explicit. The trade-off, reduced ability to generalise to entirely novel patterns, is accepted by design and discussed in Section 13.

The analyzer scores an email along three independent axes:

- **Header analysis:** authentication results (SPF, DKIM, DMARC), identity alignment, and sender-spoofing indicators.
- **Link analysis:** look-alike (typosquatted) domains via edit distance, brand misuse, raw-IP destinations, and deceptive anchor text.
- **Language analysis:** manufactured urgency, account threats, credential lures, and other social-engineering wording.

A scoring module combines the three sub-scores into one risk rating, and a reporting module renders an explainable HTML report.

Report structure. Section 3 outlines the threat model and the risk indicators the analyzer targets. Section 4 describes the datasets. Section 5 presents the overall architecture and data model. Sections 6 to 11 detail the six modules. Section 12 reports the

experimental evaluation on the full corpus. Section 13 discusses the fairness constraints that govern the false-positive rate and the residual limitations, and Section 14 concludes.

3 Threat Model and Risk Indicators

The analyzer assumes a static, *offline* setting: it inspects a single email file (an `.eml` message or a raw header-bearing message) without network access, sandbox detonation, or reputation lookups. This constraint is realistic for an automated first-pass triage filter and keeps the tool self-contained and reproducible.

Within this setting, phishing emails tend to leave one or more of the following observable traces, which the modules are designed to detect:

- **Authentication failures and identity misalignment.** A forged sender frequently fails SPF/DKIM/DMARC checks (when those are present), or shows a mismatch between the visible `From` domain and the envelope sender, DKIM signing domain, or reply address.
- **Sender impersonation.** A display name that claims a well-known brand the real domain does not back up, or an organisational/notification name sent from a free webmail account.
- **Deceptive links.** Look-alike domains (`paypa1.com`), brand names planted in attacker-owned domains (`paypal-security.com`), raw-IP hosts, the `user@host` credential trick, and anchor text that disguises the true destination.
- **Coercive language.** Urgency, threats of account closure, requests to verify credentials or sensitive data, and reward bait.

No single indicator is conclusive in isolation, as legitimate mail can exhibit any one of them. The scoring model (Section 10) is therefore designed to weight a decisive signal appropriately while rewarding the corroboration of multiple independent indicators.

4 Datasets

Two real-world corpora are used: one of phishing emails and one of legitimate mail acting as the control set.

Role	Dataset	Count	Notes
Phishing	Phishing Pot	8,608	Real phishing with full headers
Legitimate	SpamAssassin `easy_ham`	2,499	Ordinary / list mail
Legitimate	SpamAssassin `hard_ham`	251	Harder, marketing-like mail

Table 1: Datasets used for evaluation (counts after skipping a handful of unreadable files).

The **Phishing Pot** corpus consists of real phishing samples collected via honeypot mailboxes, retaining the complete header set (including `Authentication-Results`), which is essential for header analysis. The corpus is heavily multilingual, with a strong presence of Portuguese, Spanish, and German lures.

The **SpamAssassin ham** corpus is used as the legitimate control. A crucial property, exploited throughout the design, is that this mail predates the widespread deployment of email authentication (it is largely from 2002 and earlier). Consequently, the *absence* of SPF/DKIM/DMARC headers must never be treated as risk: doing so would penalise every legitimate control email. This observation drives the central fairness constraints discussed in Sections 7 and 13. The choice of SpamAssassin ham over the Enron corpus was deliberate: it provides the same pre-authentication fairness profile while being a more diverse mix of personal, mailing-list, and marketing mail.

5 Overall Architecture

The analyzer is organised as a linear pipeline of six modules around a shared data model. The parser produces a normalised `ParsedEmail`, each of the three analysis modules consumes it and returns a uniform `ModuleResult`, the scoring module folds these into an `OverallResult`, and the reporting module renders it. Treating every analysis module uniformly (a 0 to 100 sub-score plus a list of explainable findings) is what allows the combiner and the report to handle all three angles identically.

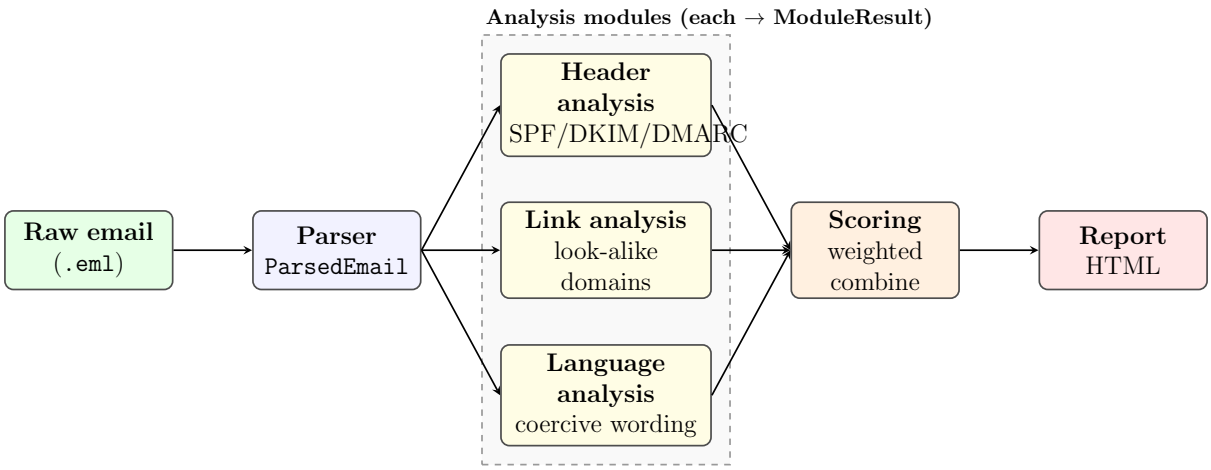

Figure 1: Overall pipeline: a single `ParsedEmail` feeds three independent analysis modules whose uniform results are combined into one risk rating and rendered as an explainable report.

5.1 Shared data model

Four small data structures form the common vocabulary across all modules:

- `ParsedEmail`: a decoded, format-independent view of the message (addresses, subject, authentication headers, body text, links, anchors).
- `Finding`: one explainable observation consisting of a stable code, a title, the concrete evidence, a severity, and the `points` it contributes to its module's score.
- `ModuleResult`: the output of one module, a 0 to 100 score plus its list of findings and raw facts.

- `OverallResult`: the final verdict, comprising the combined score, the risk band, and the per-module breakdown.

6 Module 1: Email Parsing

The parser turns raw message bytes into a clean `ParsedEmail`. It is intentionally *analysis-free*: it only decodes and normalises, so that every downstream module works from a well-formed representation regardless of on-disk quirks. Real datasets exercise many edge cases, all of which the parser handles gracefully: RFC 2047 encoded headers, base64 / quoted-printable bodies with arbitrary charsets, multipart messages, malformed or empty addresses (e.g. `From: "Facebook" <>`), and messages with no MIME structure at all.

A subtle but important case is the "comma-in-display-name" spoof, in which a header such as `Temu, jehd <service@stayfriends.de>` is parsed by the standard library as *two* mailboxes: a phantom `Temu` with no domain and the real address. The parser selects the first mailbox that actually has a domain and folds the phantom fragments back into the display name, so that downstream brand-spoof checks see the intended display name and domain. The parser additionally extracts `(anchor text, href)` pairs from HTML so that the link module can detect text that disguises its destination.

7 Module 2: Header Analysis

The header module scores the trustworthiness of an email's headers along three sub-angles: *authentication* (did SPF/DKIM/DMARC actually pass?), *identity alignment* (do the `From`, `Return-Path`, `Reply-To`, and DKIM signing domains agree?), and *sender sanity* (empty or forged `From`, brand impersonation in the display name, replies diverted to free webmail).

7.1 The fairness rule

The single most important design decision is that the **absence** of authentication headers contributes **zero** points. Because the legitimate control set predates SPF/DKIM/DMARC, penalising "no SPF result" would flag every legitimate email and destroy the evaluation. Only authentication results that are *present and failing* add risk. The same reasoning makes a Message-ID domain that differs from the `From` domain purely informational (legitimate MTAs and ESPs routinely generate their own Message-IDs).

```
def _check_authentication(result, auth, has_auth_headers):
    if not has_auth_headers:
        # Cannot verify: treat as neutral, do NOT penalise (ham-
   friendly).
        _emit(result, "AUTH_ABSENT", "No authentication results",
              "Sender authenticity could not be verified from headers."
   ,
              points=0, severity="info")
        return
    # ... only verdicts that are present and != "pass" add points ...
```

Listing 1: Authentication is scored only when present, while absence is neutral.

7.2 Sender-identity checks

Beyond authentication, the module flags a display name that names a known brand the real domain does not support (e.g. a "PayPal" display name on an unrelated domain), a display name that embeds an address or URL, and a particularly common bulk-phishing pattern: an *organisational or notification display name sent from a free webmail account* (a real bank never emails customers from `gmail.com`). The organisational-name heuristic is multilingual, matching keywords such as `banco`, `conta`, `seguranca`, and `atendimento`, reflecting the corpus.

7.3 List/bulk awareness

Mailing lists and Email Service Providers legitimately rewrite the envelope sender (`Return-Path`) and point replies at a list address. A naive "`Return-Path` domain $\neq$ `From` domain" rule therefore over-flags legitimate list traffic. The module detects list/bulk mail (via `List-*` and `Precedence` headers) and *suppresses* these weak mismatch signals for such messages, while keeping the strong variant (`Reply-To` diverted to free webmail) at full weight. This single rule is responsible for a large share of the final false-positive reduction (Section 13.1).

8 Module 3: Link Analysis

Phishing almost always requires the victim to click something, so the links are often the most reliable signal. The module de-duplicates URLs by host (so a link repeated 40 times is scored once) and evaluates several sub-angles.

8.1 Look-alike (typosquatted) domains

A registered domain that is a near-miss of a known brand domain is flagged using Levenshtein edit distance on the *registrable name*, augmented with homoglyph folding (so `paypa1.com` and `amaz0n.com` resolve onto the brand). A critical lesson from evaluation is that edit distance on *short* brand names is meaningless: `msnbc` is two edits from `hsbc`, and `ft` is two edits from `att`, yet both are legitimate. The matcher therefore (i) ignores brand names shorter than five characters, (ii) permits a distance of two only for names of length eight or more, and (iii) requires the candidate and brand to be of similar length.

Algorithm 1 Look-alike domain detection (guards against short-name noise)

function LOOKALIKE(rd) ▷ rd = registered domain of a link host
  $name \leftarrow$ label of rd before its suffix
  if $|name| < 5$ **then**
    return $(\emptyset, \infty)$
  end if
  for all brand b with $|b| \geq 5$ **do**
    if $b = name$ **or** $\,||b| - |name|\,| > 1$ **then**
      continue
    end if
    if CANONICAL(b) = CANONICAL($name$) **then**
      return $(b, 1)$ ▷ homoglyph swap, e.g. amaz0n
    end if
    $allowed \leftarrow 2$ **if** $|b| \geq 8$ **else** 1
    $d \leftarrow$ LEVENSHTEIN($name, b, allowed$)
    if $1 \leq d \leq allowed$ **then**
      record (b, d)
    end if
  end for
  return closest recorded (b, d), or $(\emptyset, \infty)$
end function

8.2 Other link signals

- **Brand misuse (combosquatting):** a brand name combosquatted into the attacker-owned registrable label (`paypal-security.com`). The check is restricted to the registered label and *not* to arbitrary subdomains, because a brand word as a subdomain of an unrelated legitimate domain (`outlook.4team.biz`, a real product) is benign.
- **Deceptive destinations:** raw-IP hosts, the `user@host` credential trick, and punycode/IDN homographs.
- **Anchor/href mismatch:** visible link text that names a trusted brand domain while the link opens a different one. This is restricted to *known-brand* visible text, because generic "reads a.com → goes to b.com" is extremely common in legitimate bulk mail (affiliate and click-tracking redirects).
- **Weak signals:** URL shorteners, abuse-prone TLDs, and deeply nested subdomains (low points each).

9 Module 4: Language Analysis

The language module scores the *wording* of the subject and body for the social-engineering patterns phishing relies on. Because the legitimate control set is full of ordinary words such as "password", "verify", and "click" used innocently in technical discussion, the rules are built around *multi-word phrases* ("verify your account", "your account has been

suspended") rather than bare keywords. Matches are grouped into categories, each with a points *cap*, so that one long email cannot inflate the score by repetition.

Category	Example phrases	Cap
Urgency	"act now", "within 24 hours", "final notice"	18
Account threat	"account suspended", "unusual activity"	20
Credential	"verify your account", "confirm your identity"	22
Sensitive info	"social security", "card number", "OTP"	20
Reward bait	"you have won", "claim your prize"	16
Non-English	"conta suspensa", "ihr konto", "haga clic"	18

Table 2: Language categories (abridged). A synergy bonus rewards messages that stack several distinct categories, the hallmark of phishing.

Two further refinements improve discrimination. First, a **synergy bonus** is added when three or more distinct manipulation categories fire together, since any single category is weak evidence but their combination is highly phishing-specific. Second, because the corpus is heavily multilingual, a **non-English** category covers high-value Portuguese, Spanish, and German lures, and since the control set is English, these phrases add detection without risking false positives.

10 Module 5: Risk Scoring

The combiner folds the three module sub-scores into one 0 to 100 risk score. A plain weighted average under-rates a single decisive signal: a blatantly brand-spoofed sender (header = 90) with no links or coercive wording would average down to "medium", which is wrong. Since phishing frequently shows exactly *one* strong angle, the overall score leans on the strongest module and adds the weighted average as corroboration:

$$\text{overall} \;=\; 0.85 \cdot \max_i(\text{score}_i) \;+\; 0.50 \cdot \frac{\sum_i w_i \cdot \text{score}_i}{\sum_i w_i}, \qquad \text{clamped to } [0, 100].$$

The peak term gives sensitivity to a self-sufficient indicator, while the average term rewards corroboration and keeps the legitimate control set, where every module is near zero, safely low. The per-module weights w_i favour the signals that are hardest to forge.

Module	Weight w_i
Header	0.40
Link	0.35
Language	0.25

Band	Score range
low	0 to 24
medium	25 to 49
high	50 to 74
critical	75 to 100

Table 3: Per-module weights (left) and risk-band cut-offs (right).

11 Module 6: Reporting

The reporting module renders an `OverallResult` as a single, self-contained HTML file (inline CSS, no external assets or JavaScript) so that it opens anywhere and is easy to

submit. The report is deliberately *explainable*: it shows the overall band and score, each module's contribution as a labelled bar, and every individual finding with the concrete evidence that produced it. A command-line interface (`python -m phish_analyzer mail.eml --html report.html`) exposes the same pipeline for interactive use.

12 Experimental Evaluation

The analyzer is evaluated on the full corpus of Table 1. An email is counted as "flagged" when its band is *medium* or above. The evaluation harness (`scripts/evaluate.py`) reports a confusion matrix and the standard metrics. Processing is single-threaded and dependency-free, on the order of a few milliseconds per email.

	Predicted phishing	Predicted legit
Actual phishing	5,773 (TP)	2,835 (FN)
Actual legit	12 (FP)	2,738 (TN)

Table 4: Confusion matrix at decision boundary = *medium*.

Metric	Value
Detection rate (recall)	67.07%
False-positive rate	**0.44%**
Precision	99.79%
Specificity	99.56%
Accuracy	74.93%
Balanced accuracy	83.31%
F_1 score	80.22%
Matthews correlation (MCC)	0.571

Table 5: Headline metrics on the full corpus (8,608 phishing, 2,750 legit). Accuracy is discussed and contextualised in Section 12.1.

The score distributions are cleanly separated. The mean overall score is 46.1/100 for phishing versus 0.8/100 for legitimate mail, and *no* legitimate email reached the *high* or *critical* band.

Band	Phishing	Legitimate
low	2,835 (32.9%)	2,738 (99.6%)
medium	1,970 (22.9%)	12 (0.4%)
high	1,842 (21.4%)	0 (0.0%)
critical	1,961 (22.8%)	0 (0.0%)

Table 6: Score distribution by band for each class.

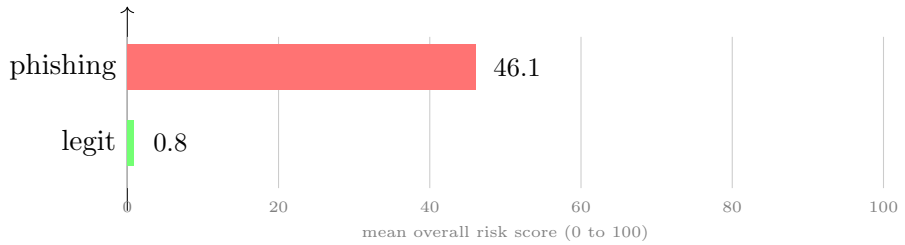

Figure 2: Mean overall risk score: phishing vs. legitimate mail. The separation is what enables a high detection rate at a very low false-positive rate.

12.1 Interpreting the confusion matrix and the accuracy figure

The confusion matrix in Table 4 shows that the analyzer's errors are overwhelmingly of a single type. Of the 2,847 misclassified emails, 2,835 (99.6%) are false negatives (phishing rated below *medium*) and only 12 are false positives (legitimate mail flagged). The system therefore almost never misjudges legitimate mail (12 out of 2,750, a 0.44% false-positive rate), but it misses about a third of the phishing (recall 67.1%). The reported accuracy of 74.9% is consequently driven almost entirely by missed phishing, not by mistakes on legitimate mail.

This makes raw accuracy a misleading summary of the detector's quality, for two reasons.

Accuracy depends on an arbitrary class balance. Accuracy can be written as

$$\text{accuracy} = \text{TPR} \cdot p + \text{TNR} \cdot (1 - p),$$

where p is the proportion of phishing in the evaluated set, TPR is the true-positive rate (recall, 67.1%) and TNR is the true-negative rate (specificity, 99.6%). Our corpus is 75.8% phishing, a ratio chosen to stress-test the detector against as many attacks as possible rather than to mirror a real mailbox. Because the true-negative rate is far higher than the true-positive rate, accuracy climbs steeply as the phishing prior falls. Rather than only computing this from the identity above, Table 7 demonstrates it directly on the data: all 2,750 legitimate emails are kept and the real phishing verdicts are randomly sub-sampled to each target rate (averaged over 2,000 draws, produced by `scripts/realistic_inbox.py`). The measured accuracy of the *same* detector rises to between 96% and 99% on a realistic inbox, while the false positives stay at the real 12 out of 2,750. The 74.9% headline is therefore an artefact of a deliberately phishing-heavy benchmark, not an intrinsic property of the detector.

Phishing rate	Inbox	TP	FN	FP	TN	Accuracy
75.8% (full corpus)	11,358	5,773	2,835	12	2,738	74.9%
10% (realistic)	3,056	205	101	12	2,738	96.3%
5%	2,895	97	48	12	2,738	97.9%
2%	2,806	38	18	12	2,738	98.9%
1%	2,778	19	9	12	2,738	99.2%

Table 7: The same detector measured on inboxes with realistic phishing rates, built from the real per-email verdicts: all 2,750 legitimate emails are kept and the phishing are randomly sub-sampled to the target rate, averaged over 2,000 draws. Accuracy rises to between 96% and 99% as the phishing rate approaches a realistic level, while the false positives remain the real 12 out of 2,750. This confirms empirically the analytical identity above.

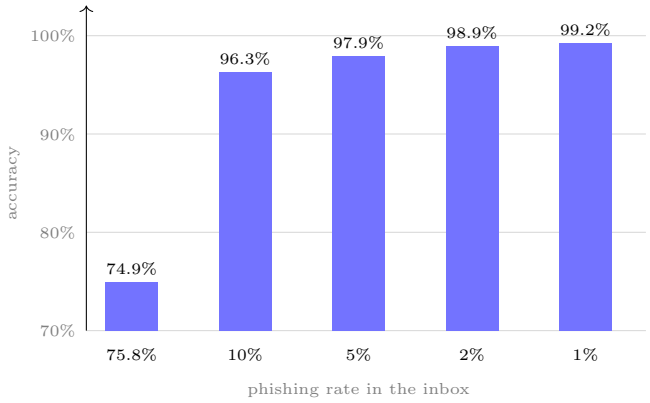

Figure 3: Accuracy of the same detector as the phishing rate in the inbox falls from the benchmark's 75.8% (leftmost bar, 74.9%) to realistic levels: on a normal inbox (1% to 10% phishing) the identical system reaches 96% to 99%. The y-axis starts at 70% so the bars are legible.

Accuracy ignores the asymmetric cost of the two errors. Accuracy weights false positives and false negatives equally, whereas in a risk-assessment setting their consequences differ sharply. A false positive quarantines a legitimate message and erodes the user's trust in the filter, while a false negative is only one missed layer of a defence-in-depth pipeline and may still be caught by other controls or by the user. The analyzer is tuned on purpose to a conservative operating point that almost never raises a false alarm, accepting lower recall in return. By treating both errors alike, accuracy penalises precisely the trade-off that the design makes deliberately.

More faithful metrics. For these reasons, the prior-independent and cost-aware metrics already reported (precision, recall, specificity and the false-positive rate) describe the system better than accuracy. Two imbalance-robust summaries confirm the picture: the balanced accuracy, $(\mathrm{TPR} + \mathrm{TNR})/2 = 83.3\%$, and the Matthews correlation coefficient, $\mathrm{MCC} = 0.57$, both of which remove the inflated weight that raw accuracy gives to whichever class dominates the test set. Finally, Table 6 shows that the missed phishing sits in the *low* band rather than just under the decision boundary, confirming that these are genuinely weak-signal messages (for instance, phishing sent from authenticated

free-webmail accounts with no scoreable link or wording), consistent with the limitations discussed in Section 13.

12.2 Per-module contribution

Table 8 shows that header authentication and sender-identity signals do most of the discriminative work, while the link and language modules add corroboration and catch cases the header module misses.

Module	Phishing mean	Legit mean
Header	42.7	0.2
Link	2.8	0.1
Language	5.3	0.5

Table 8: Mean per-module sub-score by class.

12.3 Choosing the operating point

Because the output is a graded band rather than a binary label, the decision threshold can be tuned to the use case without re-running the analysis.

Flag at…	Phishing caught	Legit flagged
medium or above	67.1%	0.44%
high or above	44.2%	0.00%

Table 9: Detection vs. false-positive trade-off at two thresholds. The *high+* boundary yields an effectively zero false-positive rate, suitable when the verdict drives an automated action rather than human review.

13 Discussion

13.1 Fairness constraints are load-bearing

The defining characteristic of this analyzer is its very low false-positive rate, and that result is not incidental: it is produced by a small set of fairness constraints, each justified by a concrete failure observed during evaluation. An early, more aggressive configuration produced a false-positive rate above 4%. Tracing every false positive to its root cause and fixing it *at the source* (rather than by raising a global threshold) brought it down to 0.44% at almost no cost to detection.

Observed false positive	Constraint introduced
Every pre-2004 legitimate email lacks SPF/DKIM/DMARC	Absent authentication adds 0 points, and only present-and-failing results score
Mailing-list mail rewrites `Return-Path` and `Reply-To`	These mismatches are weak and suppressed for detected list/bulk mail
Short brand names alias to legitimate domains (`msnbc`≈`hsbc`)	Look-alike matching requires a minimum brand-name length and homoglyph folding
A brand word as a subdomain of a real product (`outlook.4team.biz`)	Brand-misuse restricted to the attacker-owned registered label
Affiliate / tracking redirects in newsletters	Anchor mismatch restricted to known-brand visible text

Table 10: Root-cause analysis: each false-positive pattern and the constraint that eliminated it. Together these reduced the false-positive rate from $> 4\%$ to 0.44%.

13.2 Limitations

The residual misses (roughly one third of phishing at the *medium* boundary) are dominated by genuinely hard cases: phishing sent from real, fully authenticated free-webmail or compromised ESP accounts, where the lure is carried in an attachment or image rather than in scoreable header, link, or text signals. These are out of reach for a header/link/language rule engine by construction. Additionally, the public-suffix logic is a curated heuristic rather than the full Public Suffix List (sufficient for the common cases in these datasets), and the language rules, while multilingual, cannot cover every language a phishing campaign might use. These limitations are inherent to the rule-based, offline design and represent the accepted trade-off for full transparency and zero external dependencies.

14 Conclusions

This project delivers a complete, explainable, rule-based phishing email risk analyzer that scores messages from three complementary angles and combines them into a single calibrated risk rating. Evaluated on a real corpus of more than 11,000 emails, it detects 67% of phishing at a false-positive rate of only 0.44% and a precision of 99.8%, with no legitimate email ever rated *high* or *critical*. The work illustrates a theme central to risk assessment: that the quality of a detector is determined less by how aggressively it flags threats than by how carefully it avoids penalising legitimate behaviour. The most valuable engineering outcome was not the detection rate itself but the disciplined, evidence-driven reduction of false positives through a handful of fairness constraints, each traceable to an observed failure and justified on its own terms.